Quantum Cryptanalysis:

Demonstrating SHA-256 Quantum Resistance

Using Grover's Algorithm

Team Members
Aayush Patel
Dhwani Patel

Course: CPSC 4110 Quantum Algorithms
Date: 5th dec, 2025

1. Introduction

Quantum computing is getting a lot of attention these days, and for good reasons. Unlike regular computers that work with bits (which can only be 0 or 1), quantum computers use qubits that can be both 0 and 1 at the same time through something called superposition. This gives them the potential to solve certain problems much faster than any classical computer ever could. The field has been growing rapidly, with companies like IBM, Google, and various startups investing billions of dollars into building more powerful quantum machines.

One area where this matters a lot is cryptography. Many of the encryption methods we use today rely on the fact that certain math problems are really hard to solve. But quantum computers might be able to crack some of these problems much faster. This has led to concerns about whether our current security systems will hold up once powerful quantum computers become available. Security researchers and government agencies have been working on this problem for years, trying to figure out which algorithms are safe and which need to be replaced.

SHA-256 is one of the most important hash functions used in security today. It is used in Bitcoin mining, digital signatures, SSL certificates, Networking cores, and password storage. When you hash something with SHA-256, you get a 256-bit output that looks completely random. The key property is that it should be practically impossible to figure out what input created a specific hash output. This is called the preimage resistance property, and it is what makes hash functions useful for security applications.

In this project, we wanted to find out: Is SHA-256 safe from quantum computers? To answer this, we implemented Grover's algorithm, which is the best-known quantum algorithm for searching through possibilities. We ran experiments on quantum simulators and compared them to simulated real hardware to see how well it actually works in practice. Our goal was to get real numbers that we could use to make informed conclusions about SHA-256's quantum security.

Summarizing the experiments, our goal is to show case the importance of post quantum cyber security as SHA-256 is one of the best current algorithm but real world companies still operates on older RSA & various other older cryptographic algorithms which needs to update their organization’s cyber security to post quantum practices.

1.1 Objectives

We set out to accomplish the following goals in this project:

1. Build a working implementation of Grover's search algorithm from scratch using the Qiskit quantum computing framework developed by IBM.
2. Run controlled experiments comparing how classical brute-force search performs versus quantum search across multiple problem sizes.
3. Use Grover's algorithm to attack simplified versions of SHA-256 (shorten to 4 and 6 bits so we can actually simulate them on classical hardware).
4. Test how noise affects quantum algorithm performance by comparing ideal simulators with fake hardware backends like FakeManila and FakeJakarta.
5. Take what we learn from small-scale experiments and figure out what it means for real 256-bit SHA-256 security through mathematical extrapolation.

1.2 Methodology: Grover's Algorithm

Grover's algorithm was invented by Lov Grover in 1996, and it provides quadratic speedup for searching problems.

Here is what that means in simple terms: if you have N items to search through, a classical computer needs to check about N/2 items on average to find what it is looking for. Grover's algorithm only needs about the square root of N checks. This speedup comes from the quantum properties of superposition and interference.

For example, if you are searching through 1 million items, classical search needs around 500,000 checks on average. Grover's algorithm would only need about 1,000 checks. That is a huge improvement, but it is important to understand that this is not as dramatic as some other quantum speedups. Shor's algorithm, for instance, provides exponential speedup for factoring numbers, which will be much more powerful.

The algorithm works in four main steps that get repeated multiple times:

Step 1 - Initialization: We start by putting all qubits into a superposition state using Hadamard gates. This means the quantum computer is essentially looking at all possible answers at once. If we have n qubits, we create a superposition of all 2^n possible states, each with equal probability amplitude.

Step 2 - Oracle: The oracle is like a black box that can recognize the answer we are looking for. When the quantum state passes through the oracle, the correct answer gets marked with a negative phase. Everything else stays the same.

Step 3 - Diffusion: This step amplifies the marked state. It works by reflecting all the amplitudes around their average value. The result is that the correct answer becomes more likely to be measured, while wrong answers become less likely.

Step 4 - Iteration: We repeat the oracle and diffusion steps approximately sqrt(N) times, where N is the total number of items we are searching through. After the right number of iterations, measuring the qubits gives us the correct answer with high probability, typically above 95%.

One important thing about Grover's algorithm is that you cannot just keep iterating forever. If you do too many iterations, the probability of finding the right answer actually starts to decrease. It oscillates back and forth like a pendulum. So you need to stop at the right time, which requires knowing roughly how many solutions exist in your search space.

2. Experiment Design

We designed several experiments to test Grover's algorithm and understand its real-world implications for SHA-256 security. Each experiment builds on the previous one to give us a complete picture of both the potential and the limitations of quantum search algorithms.

2.1 Classical vs Quantum Search Comparison

Our first experiment directly compared classical brute-force search with Grover's quantum search. We tested three different search space sizes: 3 qubits (8 possible values), 4 qubits (16 values), and 5 qubits (32 values). For each test, we picked a random target value that changes each time you run the code, and then measured how many attempts it took to find it.

For classical search, we ran 100 trials for each problem size and averaged the results. This gives us a good estimate of the expected number of guesses needed. The classical algorithm simply shuffles all possible values and checks them one by one until it finds the target. For quantum search, we ran Grover's algorithm on the Qiskit AerSimulator and measured the success rate after the theoretically optimal number of iterations.

2.2 Hash Function Preimage Attacks

For this experiment, we created shorten versions of SHA-256 that output only 4 bits or 6 bits instead of the full 256 bits. We use the real SHA-256 function internally but only keep a small part of the output. This preserves the cryptographic properties of SHA-256 while making it possible to simulate on a classical computer.

The attack works like this: we pick a random input, hash it to get our target hash value, and then use Grover's algorithm to find that input (or any other input that produces the same hash). We verify our results by checking that the quantum algorithm actually finds valid preimages.

2.3 Iteration Optimization

Since Grover's algorithm has oscillating success probability, we ran an experiment to see this behavior directly. Using a 5-qubit search space (32 possible values), we tested iteration counts from 1 to 15 and measured the success rate at each step. This shows us exactly how over-iteration hurts performance.

2.4 Noise Impact Analysis

Real quantum computers are noisy. Gates do not work perfectly, qubits lose their quantum state over time, and measurements have errors. To understand how this affects Grover's algorithm, we ran the same 4-qubit search on multiple backends:

- Ideal AerSimulator with no noise (For theoretical performance)
- Custom noise models with error rates from 0.1% to 5%
- FakeManila & FakeJakarta backend, simulating realistic noise in quantum hardware

2.5 Experimental Parameters

Experiment	Qubits	Search Space	Iterations	Shots
Classical vs Quantum	3, 4, 5	8, 16, 32	2, 3, 4	1024
4-bit Preimage	4	16	3	2048
6-bit Preimage	6	64	6	2048
Iteration Analysis	5	32	1-15	2048
Noise Analysis	4	16	3	4096

Table 1: Summary of experimental parameters used in our tests.

3. Results

Here we present the actual results from running our experiments. All the numbers come directly from our Jupyter notebook runs with session seed 4679. (You can regenerate similar outputs again by changing session seed to 4679 in the code)

3.1 Classical vs Quantum Search Results

Our comparison showed clear quantum advantage across all problem sizes:

Qubits	Search Space	Classical Avg	Quantum Iterations	Speedup
3	8	4.7 attempts	2	2.3x
4	16	8.6 attempts	3	2.9x
5	32	15.6 attempts	4	3.9x

Table 2: Classical brute-force vs Grover quantum search comparison.

The quantum success rates were excellent on the ideal simulator. For the 3-qubit case, we achieved 94.4% success rate with just 2 iterations. The 4-qubit case gave us 96.4% with 3 iterations, and the 5-qubit case reached 100.0% with 4 iterations. These results closely match theoretical predictions.

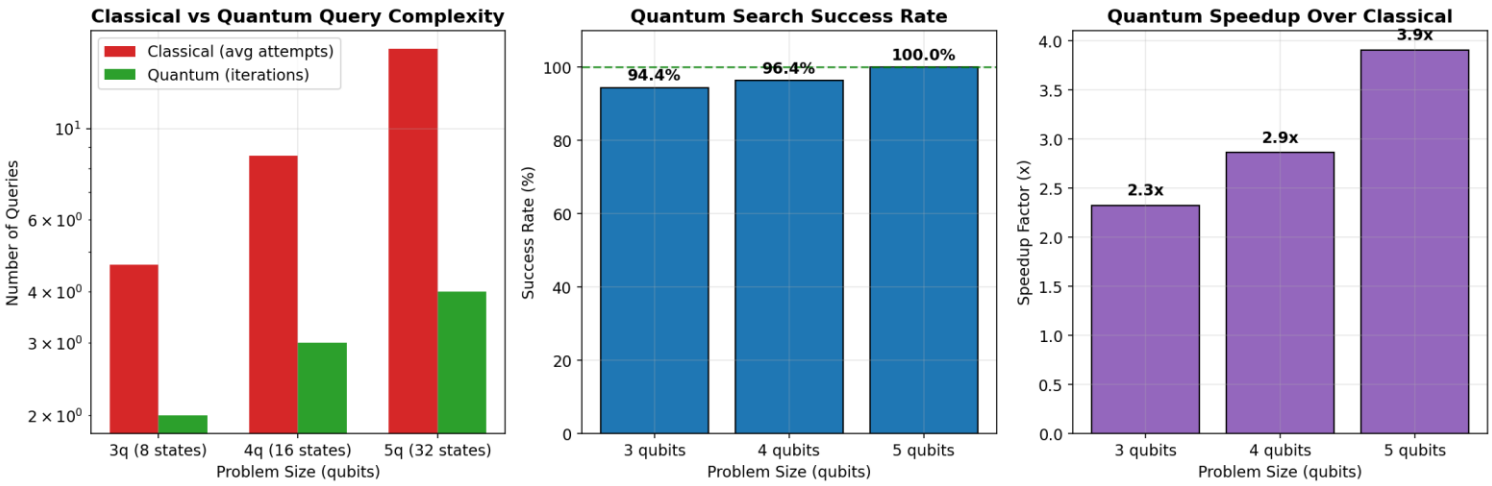


Figure 1: Classical vs Quantum search comparison showing query complexity, success rates, and speedup factors.

3.2 Preimage Attack Results

4-bit Hash Attack: We generated a random target hash value of 1. Classical analysis found that only one input (12) produces this hash. Running Grover's algorithm with 3 iterations achieved a 95.5% success rate at finding this preimage.

6-bit Hash Attack: The larger search space (64 possible inputs) had target hash 57 with 3 valid preimages (8, 35, 61). The quantum attack found them with 85.7% combined success rate using 6 iterations.

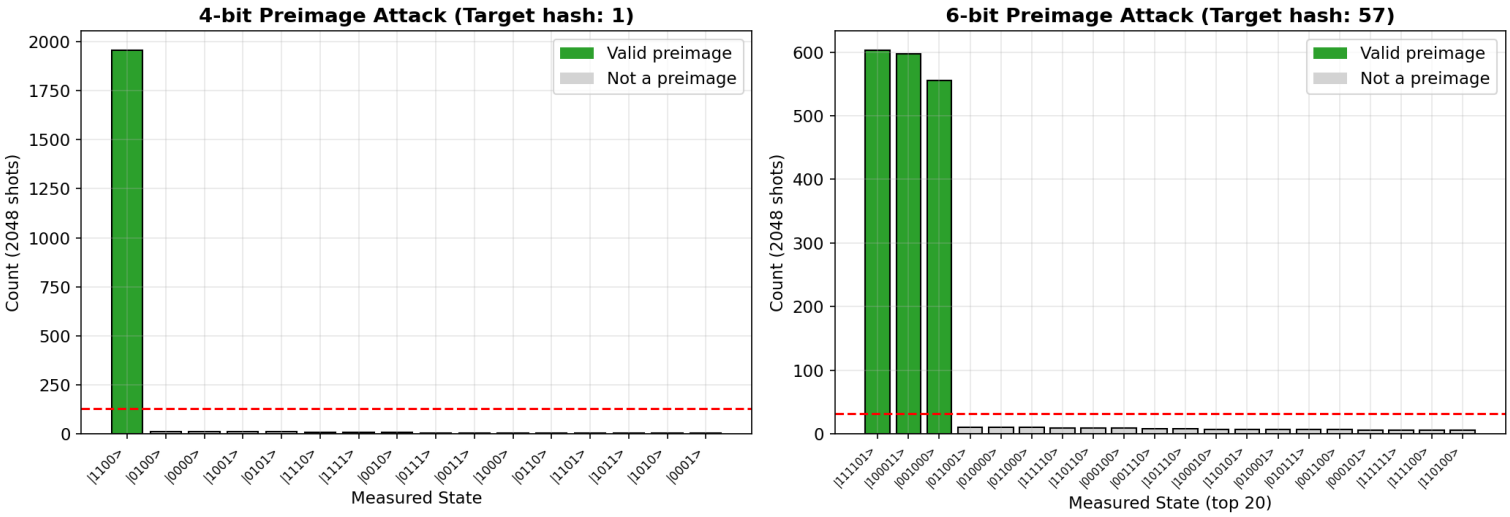


Figure 2: Preimage attack results showing measurement distributions for 4-bit and 6-bit hash attacks.

3.3 Iteration Analysis Results

Our iteration experiment clearly showed Grover's oscillating behavior. Starting from 1 iteration, the success probability increases steadily until it peaks at 4 iterations (the theoretical optimum for 32 states). After that, it drops:

- 4 iterations: 99.9% success (optimal)
- 6 iterations: 54.9% success
- 8 iterations: 1.2% success (worse than random!)
- 13 iterations: 99.2% success (oscillates back up)

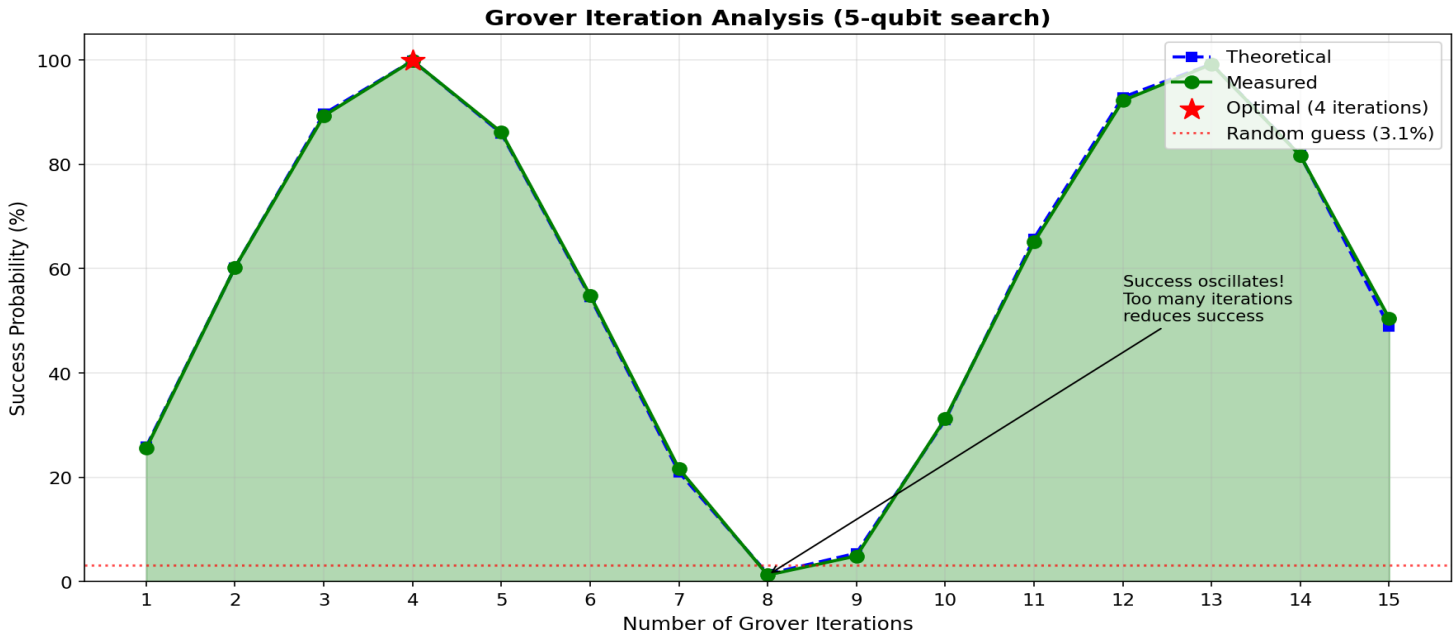


Figure 3: Grover iteration analysis showing how success probability oscillates with iteration count.

3.4 Noise Analysis Results

The noise results were eye-opening. While the ideal simulator achieved 95.9% success, adding even smallest amounts of noise caused dramatic performance drops:

Environment	Success Rate	Degradation	Circuit Depth
Ideal Simulator	95.9%	baseline	38 gates
0.1% Gate Error	46.8%	-49.2%	38 gates
0.5% Gate Error	7.2%	-92.5%	38 gates
1.0% Gate Error	6.9%	-92.8%	38 gates
FakeManila	13.5%	-85.9%	309 gates
FakeJakarta	24.0%	-75.0%	varied

Table 3: Impact of noise on 4-qubit Grover search. Random guessing would give 6.25%.

The FakeManila results show what happens on real hardware. When we convert our circuit, it expands from 38 gates to 309 gates because the hardware only supports certain gate types. This longer circuit gives noise more opportunity to accumulate.

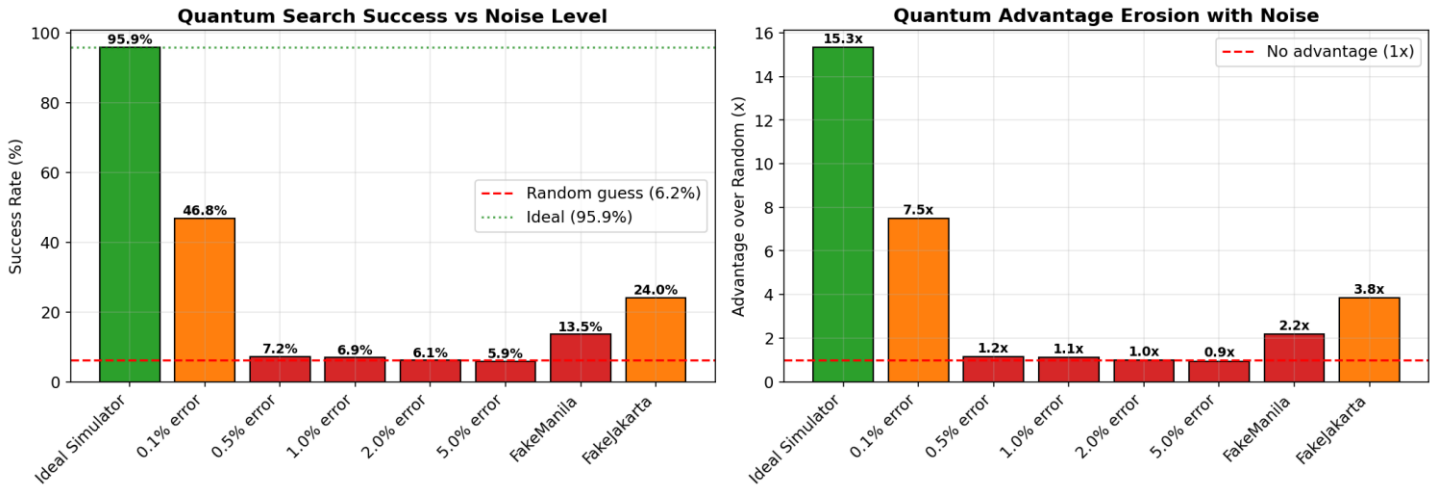


Figure 4: Noise impact analysis showing success rate degradation and quantum advantage erosion.

4. Discussion and Conclusion

4.1 Are Simulator Results Comparable to Actual Quantum Computer Runs?

Based on our experiments, the short answer is no. There is a significant gap between what we see in ideal simulation and what would happen on real hardware. Our ideal simulator achieved around 96% success rate, while FakeManila only achieved 13.5%. That is a performance drop of nearly 86%.

The main reasons for this gap are: gate errors that accumulate with each operation, decoherence where qubits lose their quantum state over time, measurement errors at the end of the circuit, and circuit expansion when converting to hardware-native gates. Anyone doing quantum computing research should test on fake backends & real hardware, not just ideal simulators.

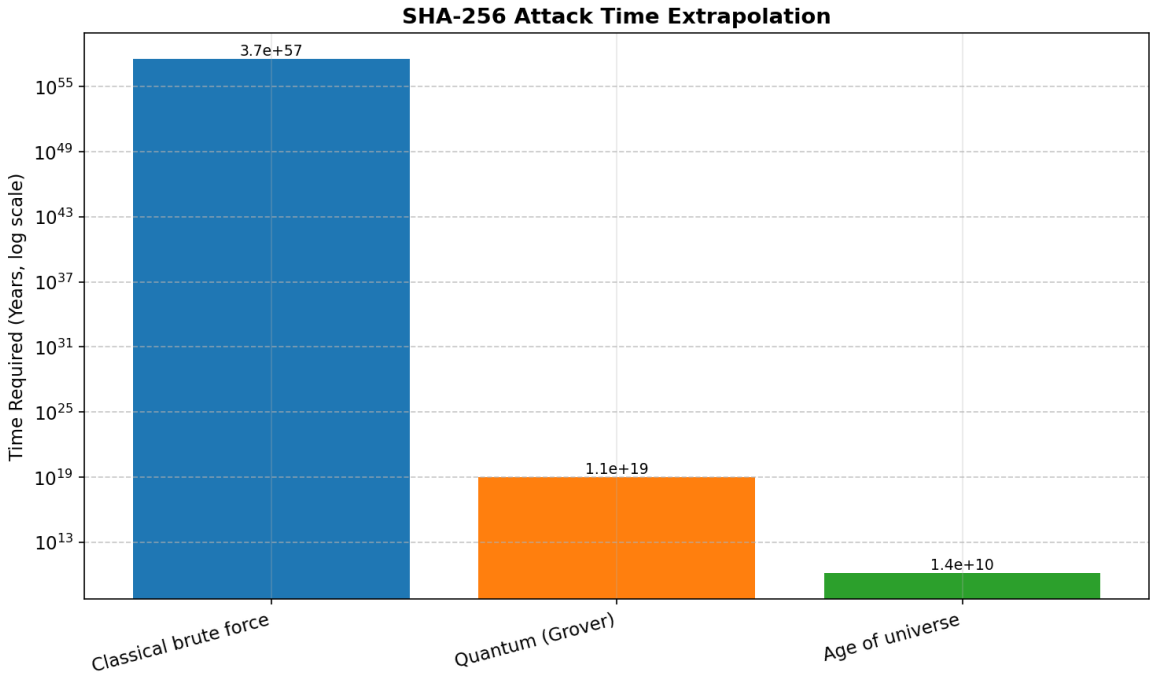
4.2 What Conclusions Can We Draw?

Conclusion 1:

Grover's algorithm really works. On ideal simulators, we consistently achieved success rates above 95%, matching theoretical predictions. The quadratic speedup is real and measurable.

Conclusion 2:

SHA-256 is quantum-safe. Even with Grover's speedup, attacking SHA-256 would require 2^128 quantum operations. This would take about 10^19 years - roughly 780 million times the current age of the universe. SHA-256 provides 128-bit security against quantum attacks.



Conclusion 3:

Noise is a dealbreaker for current hardware. Our 4-qubit experiment used only 38-92 gates. Real SHA-256 would need around 100,000 gates. With 86% degradation for just our simple circuit, scaling up would make the quantum advantage vanish entirely.

Conclusion 4:

Not all cryptography is equally safe. While SHA-256 resists Grover's quadratic speedup, RSA and ECC are vulnerable to Shor's algorithm which provides exponential speedup. Those systems need to be replaced with post-quantum alternatives.

4.3 Is Noise an Issue for Quantum Computers?

Absolutely. Noise is probably the biggest obstacle facing quantum computing today. Our experiments showed that just 0.1% gate error cut our success rate in half, and at 0.5% error we were barely better than random guessing. Current IBM quantum computers have gate error rates around 0.1-1% for single-qubit gates and 1-5% for two-qubit gates.

The solution is quantum error correction, which uses multiple physical qubits to create one reliable logical qubit. Current estimates suggest we would need 1000+ physical qubits for each logical qubit needed for serious cryptographic attacks. We are still far from that level of hardware.

4.4 Will Quantum Algorithms Impact Cryptography in the Near Future?

For hash functions like SHA-256: No. The combination of quadratic-only speedup, massive qubit requirements, and devastating noise impact means hash functions are not at practical risk.

For symmetric encryption like AES-256: Mostly safe. AES-256 provides 128-bit quantum security, same as SHA-256. The simple fix is to use AES-256 everywhere.

For public-key cryptography (RSA, ECC): Yes, this is urgent. Shor's algorithm breaks these systems entirely. NIST has already standardized new post-quantum algorithms. Organizations should be planning migration now.

5. Conclusion

This project set out to answer a simple but important question: Is SHA-256 safe from quantum computers? After implementing Grover's algorithm, running multiple experiments, and carefully analyzing the results, we can confidently answer: yes, SHA-256 remains secure.

Our key findings were:

1. Grover's algorithm works as advertised, achieving 94-100% success rates on ideal simulators and demonstrating genuine quadratic speedup over classical search.
2. The speedup is quadratic, meaning SHA-256's 256-bit security becomes 128-bit against quantum attacks - still astronomically secure.
3. Noise devastates quantum algorithm performance. Our FakeManila tests showed 86% degradation for a simple 4-qubit circuit.
4. The gap between ideal simulators and realistic hardware is massive. Research results from ideal simulations should be interpreted cautiously.
5. While SHA-256 is safe, RSA and ECC need to be replaced with post-quantum alternatives.

The broader lesson is that quantum speedup does not automatically mean classical cryptography is broken. The type of speedup matters enormously. Grover gives quadratic speedup which is not devastating for properly-sized hash functions. Shor gives exponential speedup which completely breaks RSA and ECC.

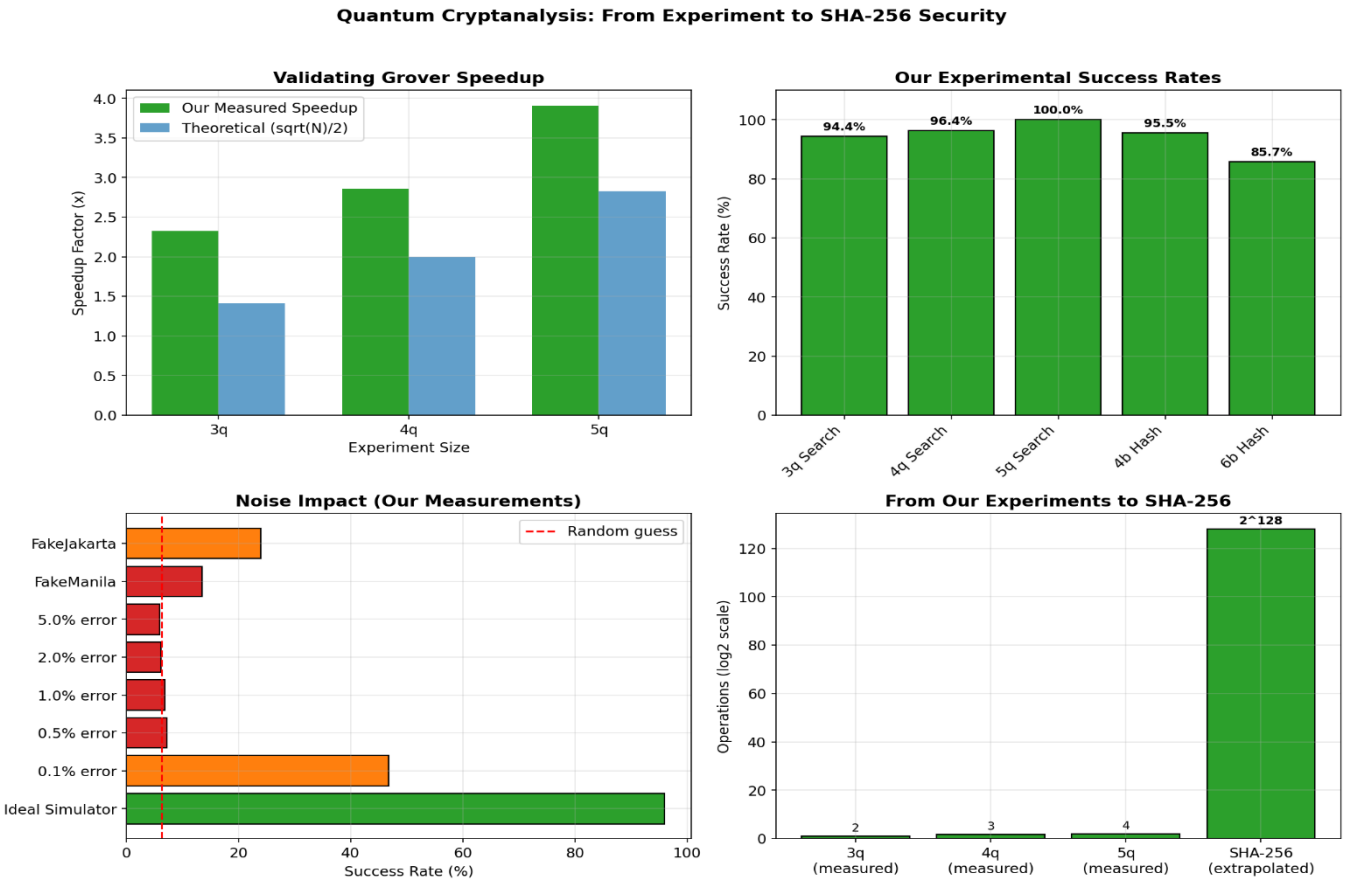


Figure 5: Executive summary showing our experimental validation of Grover speedup, success rates, noise impact, and SHA-256 extrapolation.

6. References

[1] Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC), 212-219.
[2] Amy, M., Di Matteo, O., Gheorghiu, V., Mosca, M., Parent, A., & Schanck, J. (2016). Estimating the cost of generic quantum pre-image attacks on SHA-2 and SHA-3. Selected Areas in Cryptography (SAC), 317-337.
[3] Nielsen, M. A., & Chuang, I. L. (2010). Quantum Computation and Quantum Information: 10th Anniversary Edition. Cambridge University Press.
[4] Qiskit Development Team. (2024). Qiskit: An Open-source Framework for Quantum Computing. https://qiskit.org/
[5] IBM Quantum. (2024). IBM Quantum Computing Platform and Documentation. https://quantum.ibm.com/
[6] National Institute of Standards and Technology. (2024). Post-Quantum Cryptography Standardization. https://csrc.nist.gov/projects/post-quantum-cryptography
[7] Preskill, J. (2018). Quantum Computing in the NISQ era and beyond. Quantum, 2, 79.
[8] Boyer, M., Brassard, G., Hoyer, P., & Tapp, A. (1998). Tight bounds on quantum searching. Fortschritte der Physik, 46(4-5), 493-505.